PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 07 -

OBSERVABILIDADE NO KUBERNETES

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 11
O QUE VOCÊ VIU NESTA AULA? 12
REFERÊNCIAS 13

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Prepare-se para mergulhar nos conceitos que formam a base da computação em nuvem moderna. Vamos explorar desde a sua origem e os problemas que ele se propõe a resolver, até sua arquitetura robusta e os componentes que trabalham em harmonia para automatizar a implantação, o dimensionamento e a gestão de aplicações em contêineres.

Ao final deste material, você terá uma compreensão sólida sobre o que é o Kubernetes e por que ele se tornou o padrão de fato no ecossistema DevOps e Cloud.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Durante os vídeos, você aprenderá a utilizar o kubectl para interagir com o cluster, criar Pods e Deployments para orquestrar seus containers e configurar Services e Ingress que darão vida à sua infraestrutura de aplicações distribuídas. O objetivo é que, ao final, você se sinta confiante para iniciar seus próprios projetos de orquestração de containers com Kubernetes, aplicando as melhores práticas desde o início.

Para facilitar seu aprendizado, todos os comandos e arquivos YAML utilizados nas videoaulas estarão disponíveis para consulta. Prepare seu ambiente e vamos juntos(as) explorar o poder do Kubernetes na prática.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

O Kubernetes, frequentemente abreviado como K8s, é uma plataforma de orquestração de contêineres de código aberto que automatiza o ciclo de vida de aplicações em contêineres. Em termos simples, ele gerencia um cluster de máquinas (físicas ou virtuais) e garante que as aplicações empacotadas em contêineres (como Docker) sejam executadas de forma confiável e escalável. Ele cuida de tarefas complexas como distribuição de carga (load balancing), autorrecuperação (self-healing) em caso de falhas, escalonamento automático para lidar com picos de tráfego e atualizações de software sem tempo de inatividade (zero-downtime).

Imagine que você tem uma aplicação composta por diversos microsserviços, cada um rodando em seu próprio contêiner. Gerenciar dezenas ou centenas desses contêineres manualmente seria uma tarefa hercúlea e propensa a erros. O Kubernetes entra em cena como um maestro, orquestrando todos esses contêineres para que funcionem em harmonia, garantindo que os recursos computacionais (CPU, memória) sejam utilizados da forma mais eficiente possível. Ele permite que pessoas desenvolvedoras se concentrem em escrever código, enquanto a plataforma cuida da complexidade da infraestrutura subjacente.

Importância e papel dentro do ecossistema DevOps/Cloud

No mundo DevOps e de computação em nuvem, a agilidade e a confiabilidade são reis. O Kubernetes se tornou uma peça central nesse ecossistema por ser a ponte perfeita entre o desenvolvimento (Dev) e as operações (Ops). Ele fornece uma linguagem e uma plataforma comuns, permitindo que as equipes colaborem de forma mais eficaz. Para especialistas em desenvolvimento, ele oferece um ambiente de implantação consistente, da máquina local à produção na nuvem. Para as operações, ele automatiza tarefas repetitivas, aumenta a resiliência do sistema e otimiza o uso de recursos.

O Kubernetes é o motor por trás da chamada "infraestrutura imutável" e da entrega contínua (CI/CD). Com ele, as empresas podem lançar novas funcionalidades mais rapidamente, responder a mudanças no mercado com agilidade e manter seus serviços sempre disponíveis, independentemente da escala. Ele é a base sobre a qual muitas plataformas de nuvem (PaaS - Platform as a

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Service) e arquiteturas de microsserviços são construídas, sendo hoje uma habilidade essencial para qualquer profissional da área de tecnologia.

Exemplos e analogias práticas

Uma boa analogia para entender o Kubernetes é pensar em um grande porto de contêineres. Os contêineres de navio são padronizados, o que permite que sejam transportados e empilhados de forma eficiente por guindastes e veículos. Da mesma forma, os contêineres de software (como Docker) empacotam uma aplicação e todas as suas dependências de forma padronizada. O Kubernetes atua como o sistema de gerenciamento do porto: ele decide qual guindaste (nó do cluster) deve mover qual contêiner (aplicação), onde empilhá-lo (em qual servidor executar), monitora se algum contêiner foi danificado (falhou) e o substitui automaticamente, além de gerenciar todo o fluxo de entrada e saída de navios (tráfego de rede).

Outro exemplo prático é um aplicativo de delivery de comida durante a hora do almoço. O tráfego aumenta drasticamente. Com o Kubernetes, o sistema pode ser configurado para, automaticamente, criar mais "cópias" (réplicas) da aplicação para lidar com o aumento da demanda. Assim que o movimento diminui, ele desliga essas cópias extras para economizar recursos. Tudo isso sem intervenção manual, garantindo uma experiência fluida para o usuário final.

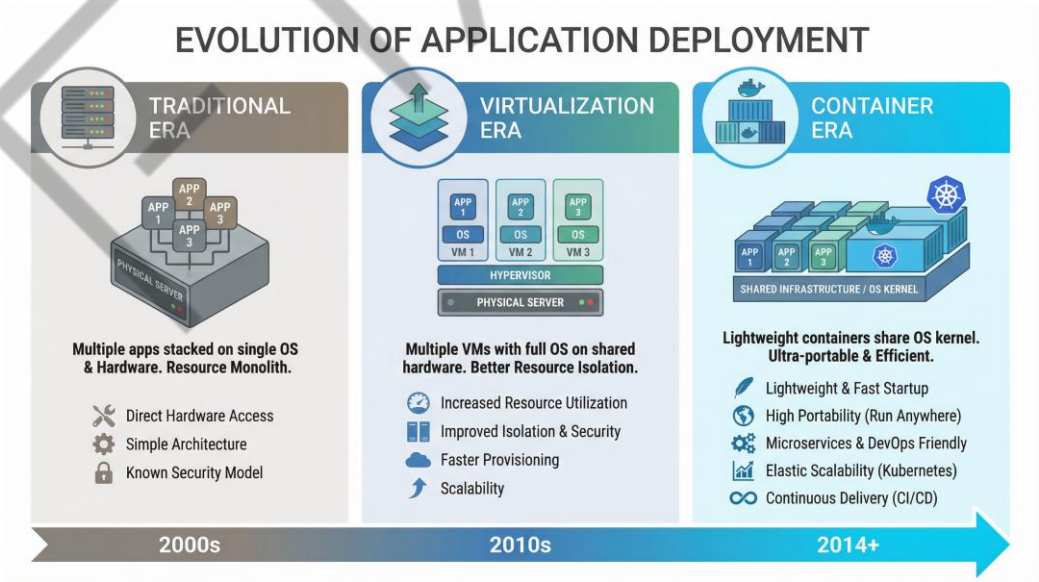


Figura 1 - Evolução dos modelos de implantação de aplicações (Deployment)
Fonte: Elaborado pelo autor com auxílio de IA (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Conceitos Fundamentais

Para dominar o Kubernetes, é crucial entender seus blocos de construção. Esses componentes e termos técnicos formam o vocabulário essencial para interagir com a plataforma e descrever o estado desejado de suas aplicações.

Principais componentes e elementos da ferramenta

Pod: é a menor e mais simples unidade de implantação no Kubernetes. Um Pod representa uma instância de uma aplicação e pode conter um ou mais contêineres que compartilham os mesmos recursos de rede e armazenamento. Geralmente, você terá um contêiner principal por Pod, mas pode adicionar contêineres auxiliares (sidecars) para tarefas como logging ou monitoramento.

Service: como os Pods são efêmeros (podem ser criados e destruídos), precisamos de uma forma estável de acessá-los. Um Service atua como um ponto de acesso de rede fixo e um balanceador de carga para um conjunto de Pods. Ele garante que, mesmo que os Pods morram e sejam recriados, a comunicação com a aplicação não seja interrompida.

Deployment: é um objeto de nível superior que gerencia o ciclo de vida dos Pods. Com um Deployment, você descreve o estado desejado: "quero 3 réplicas da minha aplicação rodando a versão 1.2". O Kubernetes então trabalha para garantir que esse estado seja sempre mantido. Ele automatiza o processo de atualização (rollout) para novas versões e pode reverter para uma versão anterior (rollback) se algo der errado.

Node: um nó é uma máquina de trabalho no cluster Kubernetes, que pode ser uma máquina virtual ou física. Cada nó executa os contêineres das aplicações e é gerenciado pelo Control Plane.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

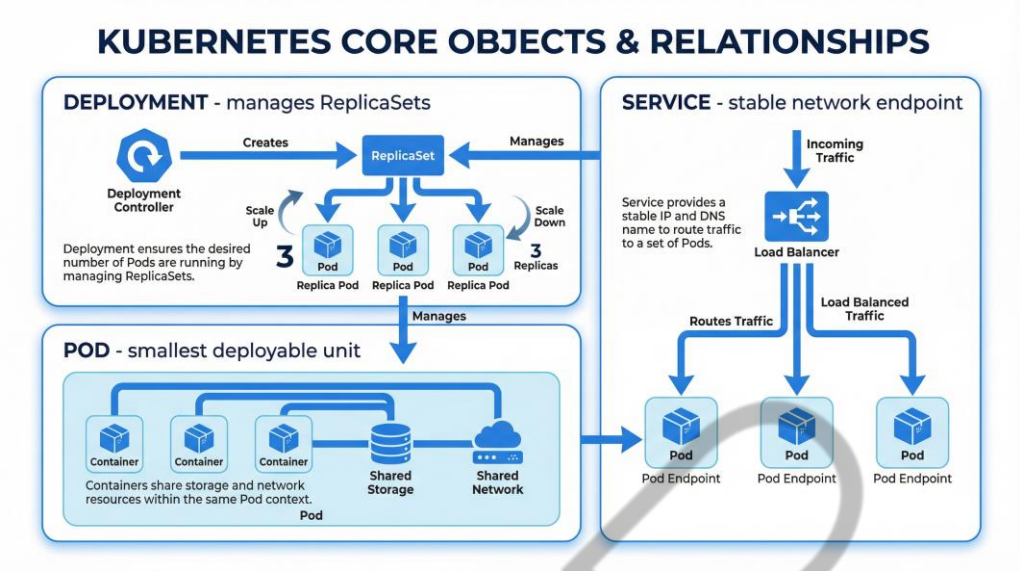


Figura 2 - Objetos principais do Kubernetes e seus relacionamentos
Fonte: Elaborado pelo autor com auxílio de IA (2026)

Termos técnicos e suas funções

Control Plane (Plano de Controle): é o cérebro do cluster. É composto por um conjunto de componentes que tomam decisões globais sobre o cluster (por exemplo, agendamento de Pods) e detectam e respondem a eventos do cluster.

kubelet: um agente que roda em cada nó do cluster. Ele garante que os contêineres descritos nos Pods estejam em execução e saudáveis.

kubectl: a ferramenta de linha de comando (CLI) para interagir com o cluster Kubernetes. Com o kubectl, você pode implantar aplicações, inspecionar e gerenciar recursos do cluster e visualizar logs.

Fluxo básico de funcionamento

O fluxo de trabalho no Kubernetes é declarativo. Você não diz ao Kubernetes como fazer algo, mas sim o que você quer como resultado final. Um desenvolvedor ou operador define o estado desejado da aplicação em um arquivo de manifesto (geralmente em formato YAML). Esse manifesto é enviado para o API Server (o portão de entrada do Control Plane) usando o kubectl. O Scheduler (agendador) vê que há novos Pods para serem criados e decide em qual Node eles devem ser executados, com base nos recursos disponíveis. O kubelet em cada nó está constantemente monitorando o API Server. Quando ele vê que um Pod foi atribuído ao seu nó, ele instrui o Container Runtime (como o Docker) a baixar a imagem e

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

executar o contêiner. O Controller Manager observa o estado do cluster e trabalha para movê-lo em direção ao estado desejado, por exemplo, criando novas réplicas se um Pod falhar.

Arquitetura

A arquitetura do Kubernetes é projetada para ser robusta, escalável e resiliente. Ela é dividida em duas partes principais: o Control Plane, que atua como o cérebro, e os Worker Nodes, que são os músculos que executam as aplicações.

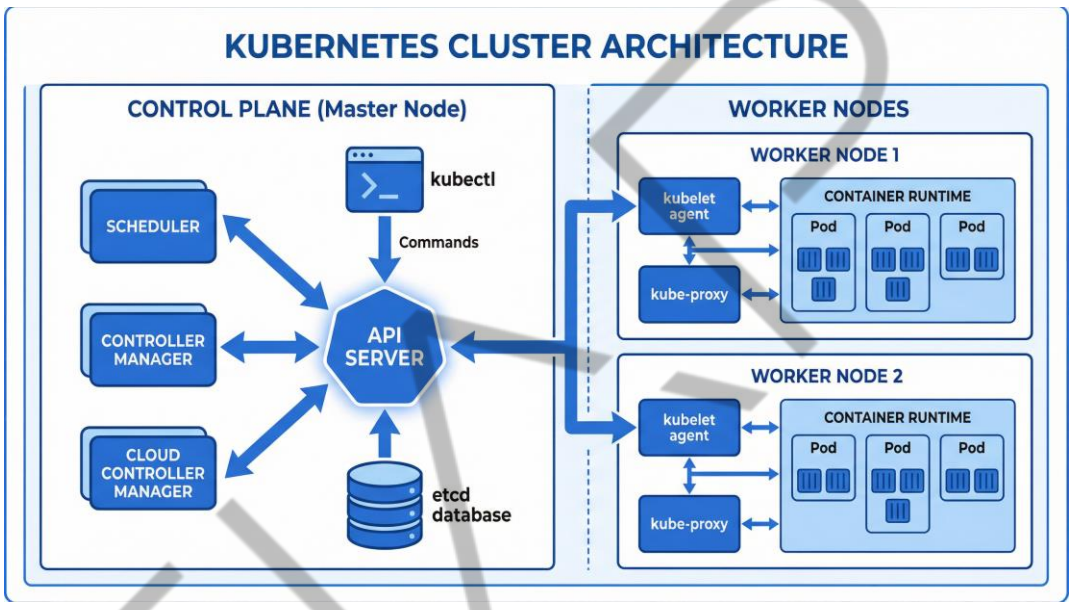


Figura 3 - Arquitetura do Kubernetes
Fonte: Fonte: Elaborado pelo autor com auxílio de IA (2026)

Descrição da comunicação entre componentes

Toda a comunicação no cluster passa pelo API Server, que é o componente central do Control Plane. Ele serve como um hub unificado, validando e processando as requisições. Usuários e ferramentas (kubectl) interagem com o cluster enviando comandos para o API Server. Os componentes do Control Plane, como o Scheduler e o Controller Manager, observam o API Server para saber quais ações precisam tomar. Eles não se comunicam diretamente entre si, mas através do estado dos objetos armazenados no etcd. O etcd é o banco de dados chave-valor que armazena de forma persistente e confiável todo o estado do cluster. O API Server é o único componente que se comunica diretamente com o etcd.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Cada kubelet (nos Worker Nodes) se registra no API Server e o monitora constantemente para receber novas instruções (manifestos de Pods) e para reportar o status do nó e dos Pods que está gerenciando. O kube-proxy, também presente em cada Worker Node, observa o API Server em busca de mudanças nos objetos do tipo Service e Endpoints, e então atualiza as regras de rede no nó para permitir a comunicação e o balanceamento de carga para os Pods.

Explicação do fluxo interno (dados, monitoramento, execução etc.)

O fluxo interno é um ciclo de reconciliação contínuo. O Control Plane está sempre comparando o estado desejado (definido nos manifestos) com o estado atual do cluster (reportado pelos kubelets). Quando há uma diferença, os controladores agem para corrigir. Por exemplo, se um nó fica offline, o Controller Manager detecta isso. Ele então marca os Pods que estavam naquele nó como "mortos" e cria novos Pods para substituí-los. O Scheduler então atribui esses novos Pods a nós saudáveis. Esse ciclo garante que o sistema seja autorreparável (self-healing) e mantenha o número desejado de réplicas da aplicação sempre em execução, sem intervenção manual.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

MERCADO, CASES E TENDÊNCIAS

Case study: Spotify - Link

Case study: BlaBlaCar - Link

Case study: Adidas - Link

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Neste material, exploramos o universo do Kubernetes, desde sua definição e propósito até sua arquitetura detalhada e aplicação prática no mercado de trabalho. Vimos como ele automatiza a gestão de aplicações em contêineres, sua importância no ecossistema DevOps e como seus componentes, como Pods, Services e Deployments, trabalham juntos.

Agora você compreende o fluxo de funcionamento de um cluster e os benefícios operacionais que ele traz para as empresas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

KUBERNETES. Kubernetes Documentation. Disponível em: https://kubernetes.io/docs/home/. Acesso em: 08 jan. 2026.

KUBERNETES. What is Kubernetes?. Disponível em: https://kubernetes.io/docs/concepts/overview/. Acesso em: 08 jan. 2026.

KUBERNETES. Kubernetes Components. Disponível em: https://kubernetes.io/docs/concepts/overview/components/. Acesso em: 08 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Kubernetes, Orquestração de Contêineres, DevOps, Cloud Native, Microsserviços, Implantação Automatizada, Escalabilidade.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com